

AARADHYA ONE

Family Spiritual Operating System

Complete Product Design Architecture, Software Architecture, Data Architecture, AI Architecture, Security Architecture & End-to-End Workflows

Blueprint objective

Design a scalable mass-market devotional ecosystem that combines personalized pooja commerce, family tradition management, verified Acharya services, temple connections, guided rituals, SacredBox replenishment, VEDA AI assistance, parent/NRI care, and circular sacred-waste recovery into one coherent platform.

Product model	Platform ecosystem + recurring commerce + marketplace + SaaS
Primary channels	Consumer App, Web, Partner App, Acharya App, Temple Portal, Admin
Suggested stack	Next.js / Flutter + Node.js/NestJS + PostgreSQL + Redis + Object Storage
Architecture style	Modular monolith for MVP -> domain services as scale demands
Core moat	Family Dharma Profile + verified ritual knowledge graph + event history + replenishment intelligence

1. Executive Architecture Summary

Aaradhya One is designed as a family-centric devotional platform. The architecture deliberately separates trusted ritual knowledge from AI-generated assistance, keeps commerce and service transactions auditable, and supports a phased path from a fast MVP to a national multi-tenant platform. The recommended initial implementation is a modular monolith with clean bounded contexts, PostgreSQL as the system of record, Redis for cache/queues/locks, object storage for media, and an event bus abstraction that can later be externalized.

Design principle: one Family Profile becomes the orchestration key connecting calendar events, ritual needs, inventory, services, payments, temple activity, memories, and circular-return rewards.

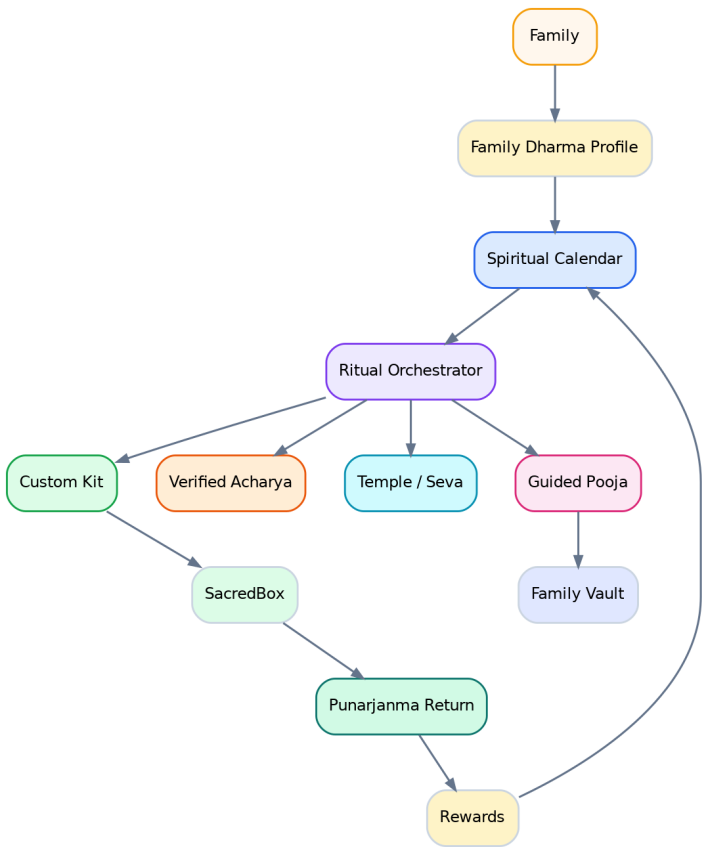


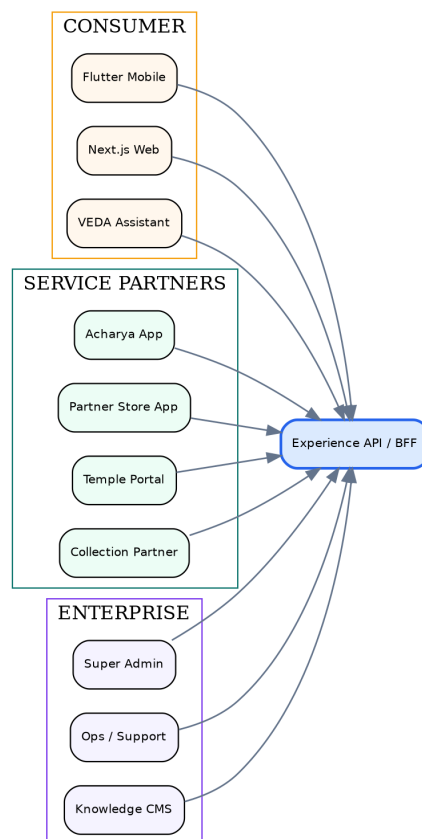
Figure 1 - Closed-loop platform architecture.

Architecture Goals

Goal	How architecture satisfies it
Personalization	Family profile, ritual rules, preference graph, event calendar, past behavior
Trust	Verified content, source metadata, human review workflow, audit history
Mass-market simplicity	Consumer UI stays task-based; complexity remains behind orchestration services
Recurring revenue	Auto-refill, memberships, parent care, NRI care, marketplace commissions
Scalability	Stateless APIs, caching, queues, partition-ready data model, CDN/object storage
Partner expansion	Tenant-aware partner, temple, vendor and Acharya portals
Safety	No autonomous religious doctrine generation; AI retrieves approved knowledge and assists orchestration

2. Product Design Architecture

The product is split into experience layers so that a user sees a simple devotional assistant while operational teams see specialized portals. Each layer uses the same identity, order, ritual and notification foundations.



Primary Product Surfaces

Surface	Core Jobs	Key UX
Consumer app	Plan rituals, buy, book, learn, manage family	Home calendar, Ready for Puja card, SacredBox, VEDA, Vault
Partner store	Fulfill local products and kits	Inventory, pick/pack, SLA, substitutions, settlements
Acharya	Manage availability and ceremonies	Calendar, service profile, booking, checklist, earnings
Temple portal	Publish official seva/products	Seva catalog, slot capacity, orders, fulfillment, reports
Collection partner	Collect eligible sacred organic waste	Pickup route, bag scan, weight, handoff, proof
Admin	Govern ecosystem	Tenants, users, content, fraud, orders, finance, analytics
Knowledge CMS	Create verified ritual guidance	Versioning, multilingual review, approvals, citations/sources

Core Consumer Navigation

- Home - today/upcoming observances, family reminders, current orders and quick actions.
- Pooja - discover by occasion, family tradition, DIY vs Acharya-assisted, and participant count.
- Store - routine essentials, verified products, personalized kits and temple-authorized items.
- Services - Acharya booking, temple seva, ceremony packages, parent/NRI care.
- SacredBox - estimated stock, scan/consume/refill, upcoming kit requirements.

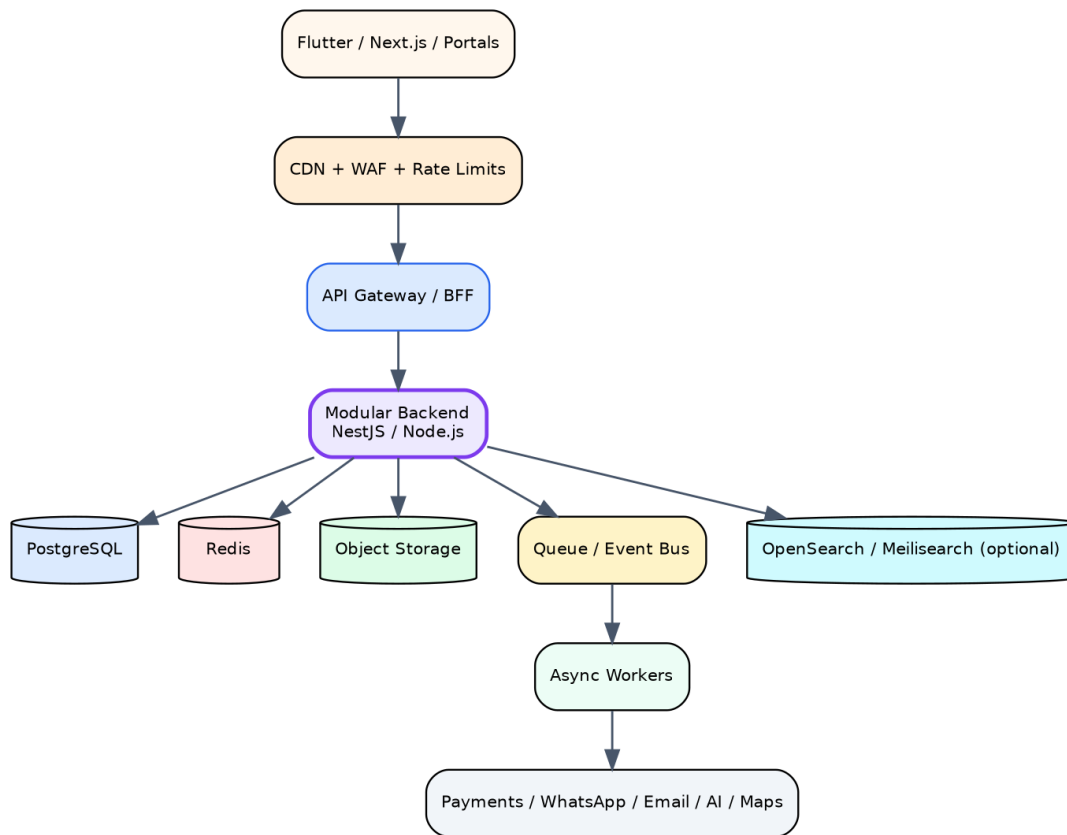
- VEDA - conversational orchestration over verified knowledge and account actions.
- Family Vault - historical rituals, media, invoices, notes, certificates and recurring observances.
- Punarjanma - eligible returns, pickups, processing status, impact and rewards.

3. Capability & Domain Architecture



4. Recommended Software Architecture

For the first commercial version, use a modular monolith instead of dozens of microservices. This reduces operational complexity while preserving strict domain boundaries. Extract services only when traffic, ownership, compliance or independent scaling justifies it.



Suggested Technology Stack

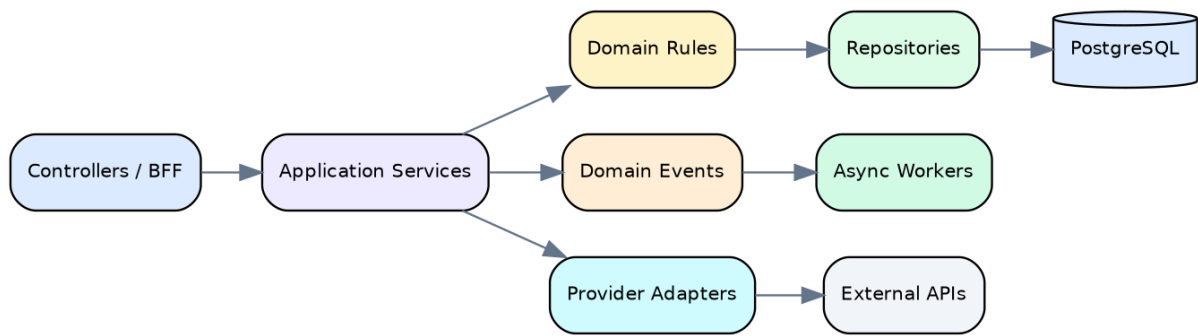
Layer	Primary choice	Alternative / note
Consumer mobile	Flutter	React Native if team already standardized on TS
Web / portals	Next.js + TypeScript	React + Vite for admin-only portals
Backend	Node.js + NestJS + TypeScript	Laravel is viable if PHP team is stronger
Database	PostgreSQL	PostGIS extension for geo queries when needed
Cache / locks	Redis	Use for sessions, OTP rate limits, carts, distributed locks
Async jobs	BullMQ initially	Kafka/RabbitMQ only when scale/use cases justify
Storage	S3-compatible object storage	Images, ritual media, invoices, reports, audit evidence
Search	Postgres FTS initially	Move catalog/location search to OpenSearch/Meilisearch later
Observability	OpenTelemetry + Grafana stack	Central logs + metrics + traces
AI	Provider-agnostic AI gateway	RAG over approved ritual knowledge; tool calling for actions

Internal Module Layout

Each backend module should own its application services, domain rules, repositories and API contracts. Cross-domain writes are performed through application services or domain events, not direct table manipulation. This makes future

service extraction safer.

5. Internal Backend Module Architecture

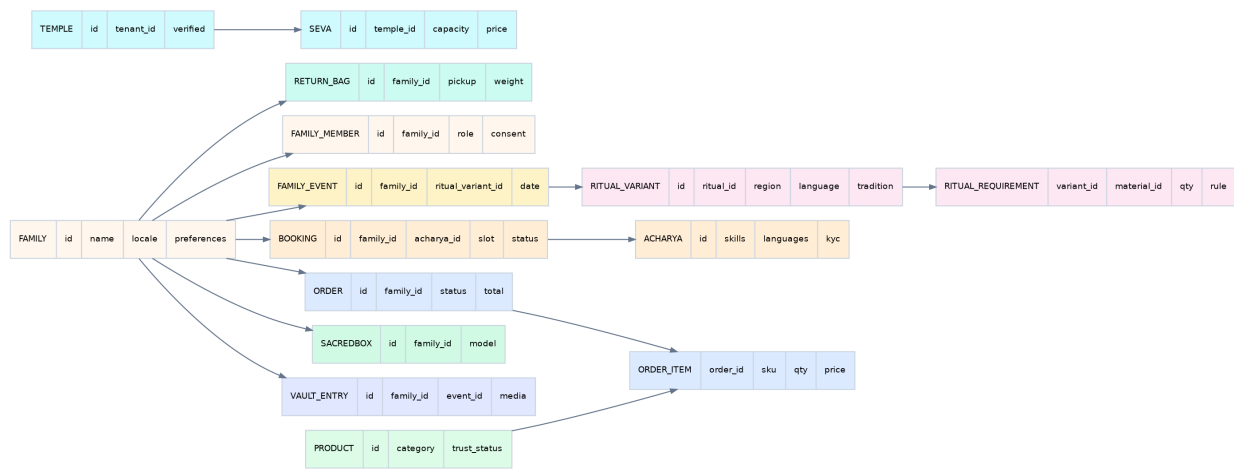


Representative Modules

Module	Key services
auth	OTP/passwordless, device sessions, refresh tokens, MFA for staff
family	households, members, sharing, consent, relationships
ritual	ritual templates, variants, requirement engine, guidance workflow
calendar	festival import, recurrence, family events, reminder scheduling
catalog	products, variants, bundles, certifications, pricing
inventory	stock by partner location, reservations, substitution policy
orders	cart, checkout, orchestration, fulfillment, cancellations
booking	Acharya discovery, slot locks, booking, reschedule, completion
temple	tenant settings, seva slots, donations where supported, prasadam
sacredbox	compartments, stock model, scan events, depletion forecast
circular	return bag, pickup, lot processing, output product traceability
finance	payment intent, refund, commission, settlement, ledger
notifications	template engine, channel router, quiet hours, delivery events
knowledge	multilingual content, source evidence, review/version workflow
ai	RAG retrieval, intent router, tool execution policy, conversation state
support	tickets, disputes, service recovery, escalations, SLA
analytics	events, funnels, cohort, demand forecasting, impact metrics

6. Data Architecture

PostgreSQL should remain the authoritative transactional store. Use schemas or strict table prefixes by bounded context, UUID/ULID identifiers, immutable ledger/event tables for money and audit, and row-level tenant scoping for temple/partner enterprise data.



Data Classification

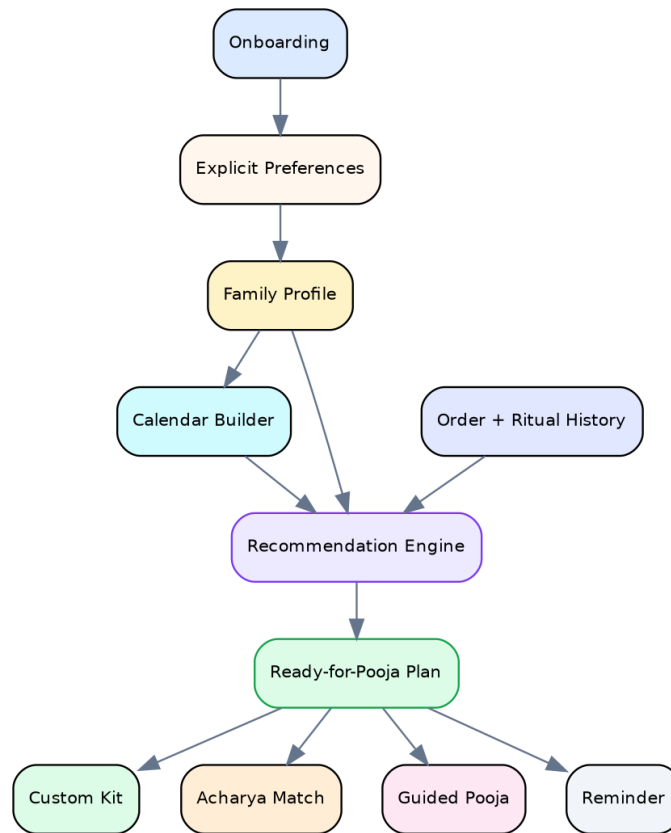
Class	Examples	Controls
Public	product catalog, public temple page	CDN cache; integrity checks
Account	name, contact, addresses	encryption at rest/in transit; least privilege
Preference	language, reminder preferences, voluntarily supplied explicit consents	explicit consent, edit/delete/export
Transactional	orders, bookings, invoices, settlements	immutable histories; reconciliation
Media	ritual photos/videos, certificates	private object ACL; signed URLs; retention controls
Trust/KYC	partner/Acharya verification documents	restricted staff roles, isolated storage prefix, audit
AI interaction	prompts, retrieved references, tool actions	redaction, short retention option, traceable source IDs

Data Rules

- Never encode caste/religious identity assumptions from names or geography. Ritual preferences must be explicitly user-provided or chosen.
- Store money as integer minor units plus currency. Maintain a double-entry ledger for settlements/rewards.
- Use soft deletion only where legally/operationally necessary; support privacy erasure for eligible personal data.
- All multi-tenant tables carry tenant_id and are scoped in repository/query layers.
- Large analytics streams are copied asynchronously into an analytical warehouse/lake when scale requires it.

7. Family Dharma Profile & Personalization Engine

The Family Dharma Profile is a user-controlled preference and history layer. It must be designed as an organizer, not an automated identity classifier. The platform learns only from selections, purchases, completed rituals, reminders and explicit feedback.

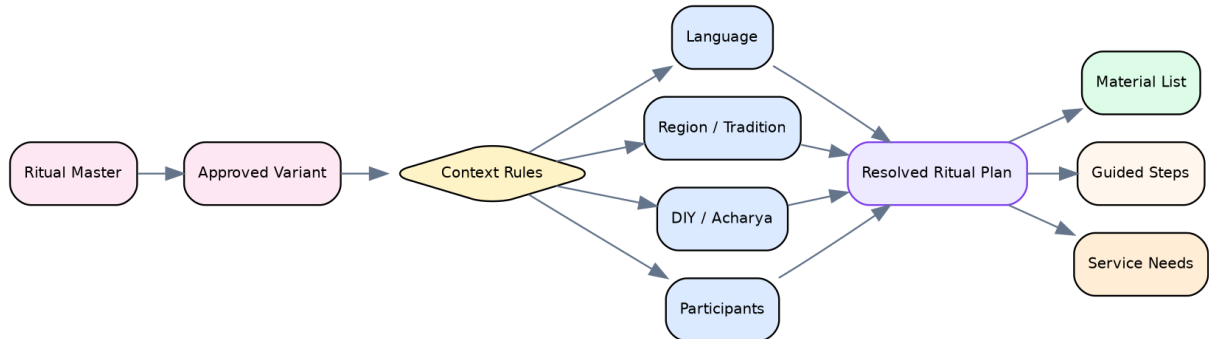


Personalization Signals

Signal	Source	Use
Explicit	user-selected language, traditions, preferred rituals	variant filtering, content language
Behavioral	products consumed, kits purchased, completed rituals	replenishment and relevance ranking
Temporal	festival/event dates, delivery lead time	preparation window and reminders
Household	members, shared responsibilities, preferred payer	family task assignment, parent-care flows
Service	favorite Acharya, ratings, completion history	repeat matching
Inventory	SacredBox scans/estimated stock	avoid duplicates, create top-up kit

8. Ritual Knowledge & Rules Engine

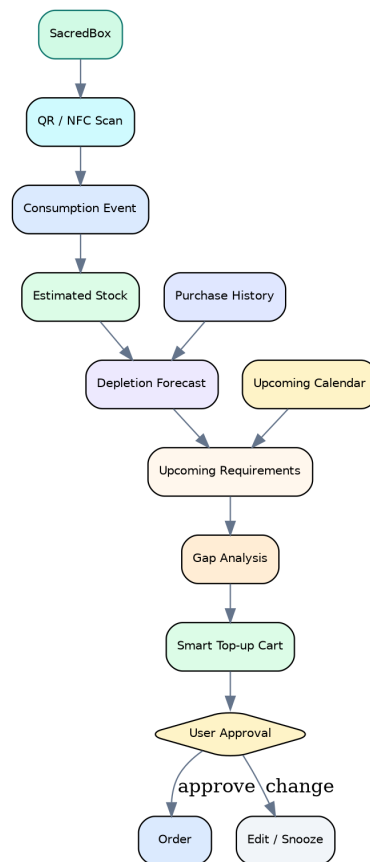
The AI must never invent ritual steps. Approved ritual variants and instructions are authored or reviewed by trusted subject-matter experts, versioned in the Knowledge CMS, and then retrieved by VEDA.



Knowledge Object Model

Object	Important fields
Ritual	canonical name, purpose/description, category, lifecycle/festival classification
Variant	language, region/tradition applicability, version, reviewer, effective date
Requirement rule	material, mandatory/optional, quantity formula, substitution constraints
Instruction step	sequence, text, audio, illustration/video, warning, completion action
Source evidence	source type, reference, author/reviewer notes, confidence/approval
Service template	duration, Acharya skill tags, preparation requirements, remote/on-site eligibility

9. SacredBox & Predictive Replenishment Architecture

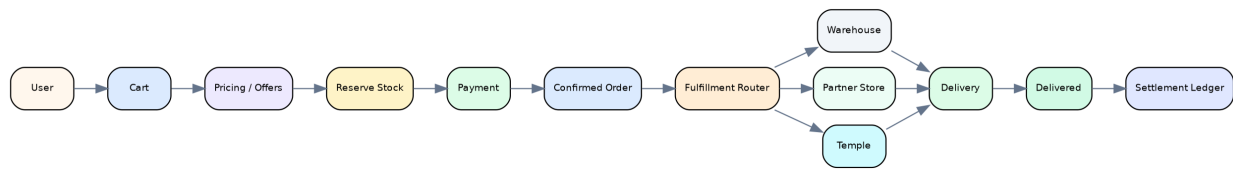


V1 to Premium Hardware Path

Stage	Implementation	Why
V1	Reusable organizer + QR labels + app-based stock estimate	Low cost, no electronics, validates behavior
V1.5	NFC tags for selected compartments	Faster scans and stronger product identity
V2 Premium	Bluetooth-enabled box with selected weight sensors	Higher-fidelity stock for high-frequency consumables
V3	Optional ambient device hub / kiosk integration	Household automation only after demand is proven

Forecasting should initially use transparent rules: purchased quantity - logged/estimated consumption + minimum safety stock. Machine learning is only introduced after sufficient real household data exists.

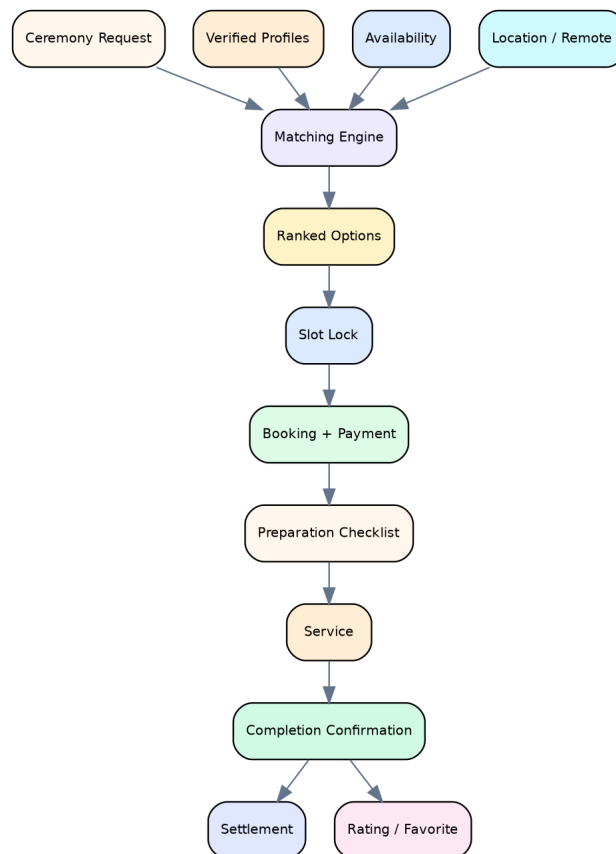
10. Commerce, Inventory & Fulfillment



Inventory Design

- SKU stock is held per fulfillment location: warehouse, partner store, temple outlet or micro-kiosk.
- Checkout reserves inventory with a short Redis-backed TTL and a database reservation record for audit.
- Substitutions are governed by product-level policy and ritual requirement rules. A cheaper substitute is not automatically acceptable for all ritual contexts.
- Bundle kits are virtual bundles whenever possible: expand to component SKUs at allocation time. This avoids duplicate inventory pools.
- Forecasting combines recurring essentials, local festival demand, partner velocity and pre-booked ceremonies.

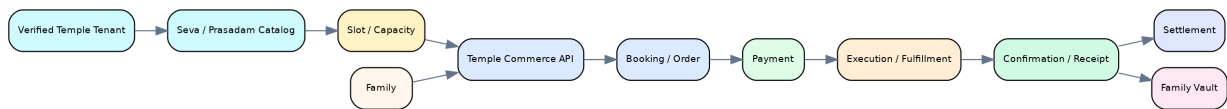
11. Verified Acharya Marketplace Architecture



Matching Factors

Factor	Examples
Skill match	ritual/service tags and verified training/experience
Language	family-selected communication language
Variant compatibility	only where explicit ritual variant requirements exist
Availability	requested date/time with travel buffers
Location	distance, service radius, transport constraints
History	family favorite / successful previous booking
Quality	completion rate, cancellation rate, verified reviews
Fairness	avoid permanent winner-takes-all ranking; introduce exposure balancing

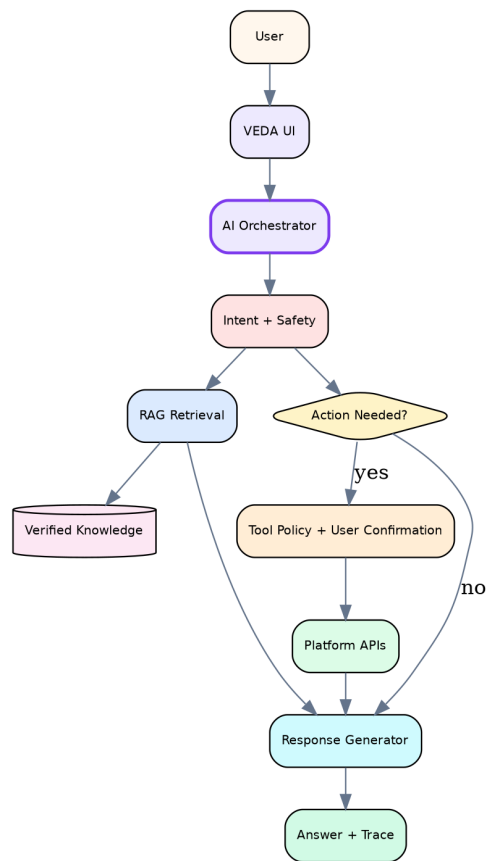
12. Temple Ecosystem Architecture



Temple Multi-Tenant Rules

- Every temple is a tenant with scoped staff, catalog, fulfillment settings, settlement account and reporting.
- Only officially verified temple content receives a verification badge.
- Capacity-based seva inventory uses slot counters with transactional locking.
- Temple-specific cancellations/refunds are configurable by policy but presented clearly before payment.
- Donation handling, tax receipts and regulatory treatment must be configured per applicable legal/accounting requirements rather than treated as ordinary e-commerce by default.

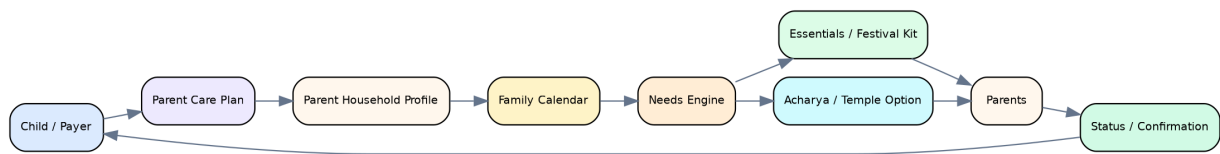
13. VEDA AI Architecture



VEDA Guardrails

Rule	Implementation
Verified ritual content only	RAG filters to approved published versions; missing information triggers a limitation response
No silent purchases/bookings	All money-moving or booking actions require explicit user confirmation
Source traceability	Store knowledge IDs/version and tool actions for each answer
Sensitive inference controls	Do not infer family identity attributes from names, location, browsing or third-party data
Prompt injection defense	Separate system/tool instructions from retrieved content; sanitize external HTML/content
Human escalation	Disputed ritual content, safety concerns, temple issues and service complaints route to support/CMS reviewers

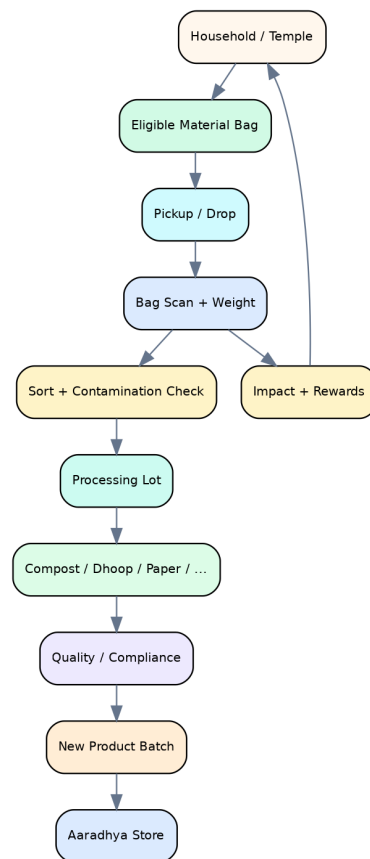
14. Parent Care & NRI Family Coordination



Consent & Control

The paying family member must not automatically gain access to all private parent data. Parent/household sharing permissions should independently control address, calendar, order history, media and Vault entries. Payment sponsorship and information access are separate permissions.

15. Punarjanma Circular Economy Architecture

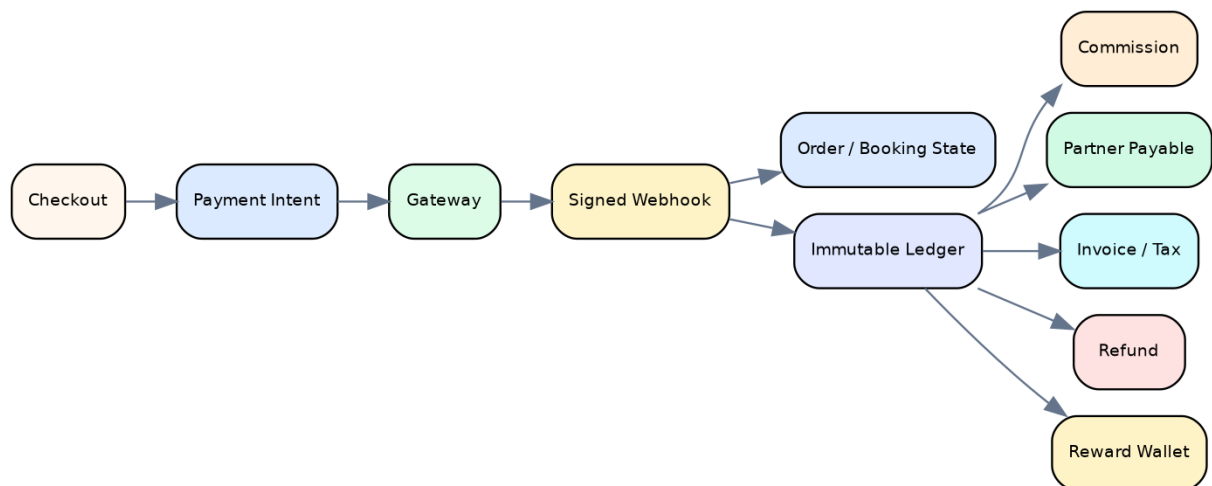


Traceability

Entity	Trace data
Return bag	source type, user/temple, pickup, weight, accepted/rejected categories
Processing lot	bags included, facility, process, timestamps, yield
Output batch	material type, test/compliance results, quantity
Finished product	input batch references where practical, manufacturing lot, expiry
Impact statement	accepted mass, diversion estimate, reward calculation methodology

Only materials supported by a validated processing SOP should be collected. The app must clearly show accepted and prohibited items to avoid unsafe contamination.

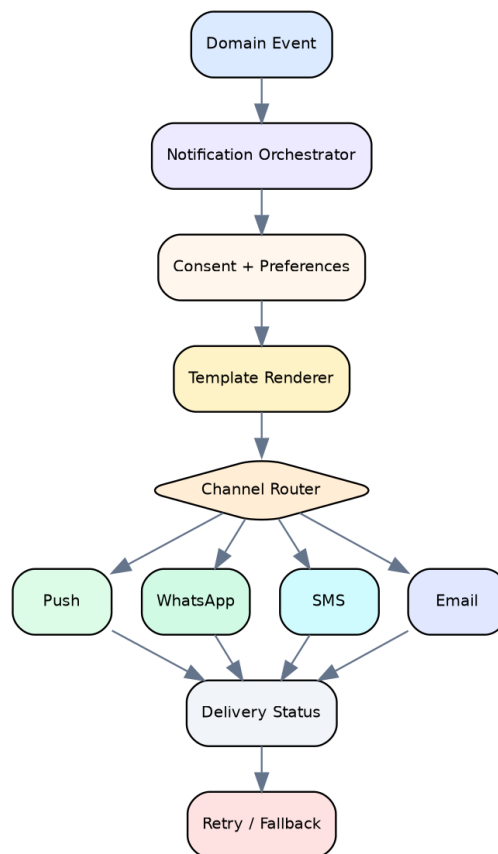
16. Payments, Rewards & Settlement Architecture



Financial Integrity Rules

- Gateway webhook processing must be idempotent. Never trust only the frontend payment success screen.
- Settlement ledger entries are append-only. Corrections use compensating entries.
- Acharya, store, temple and collection-partner payouts have separate settlement rules and statements.
- Reward points are not mixed with cash ledger balances. Define expiry, reversal and fraud rules separately.
- Reconciliation job compares gateway payments/refunds against internal intents and ledger totals daily.

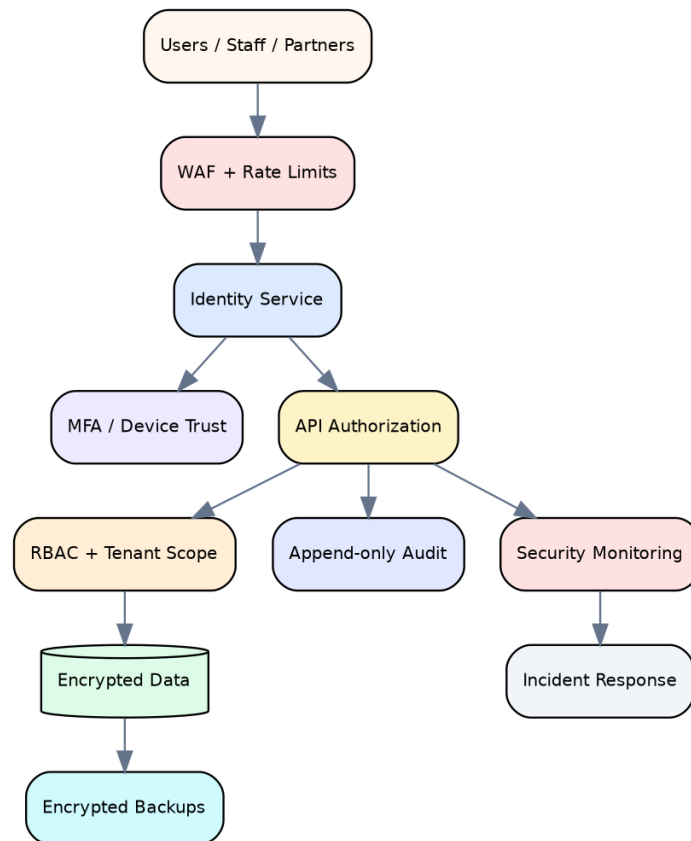
17. Notification & WhatsApp Architecture



High-value Messages

Trigger	Example
Preparation window	"Your configured festival is in 5 days; 3 materials are missing."
Order	payment confirmation, pick/pack, delivery
Acharya	booking accepted, reminder, arrival, completion
Temple	seva confirmed, prasadam dispatch, official update
SacredBox	estimated depletion with editable top-up suggestion
Parent care	payer receives fulfillment status; parent receives delivery/service message per consent
Punarjanma	pickup reminder, accepted weight, reward, impact
Security	new device, staff MFA, suspicious login, permission change

18. Security & Privacy Architecture



Role Model

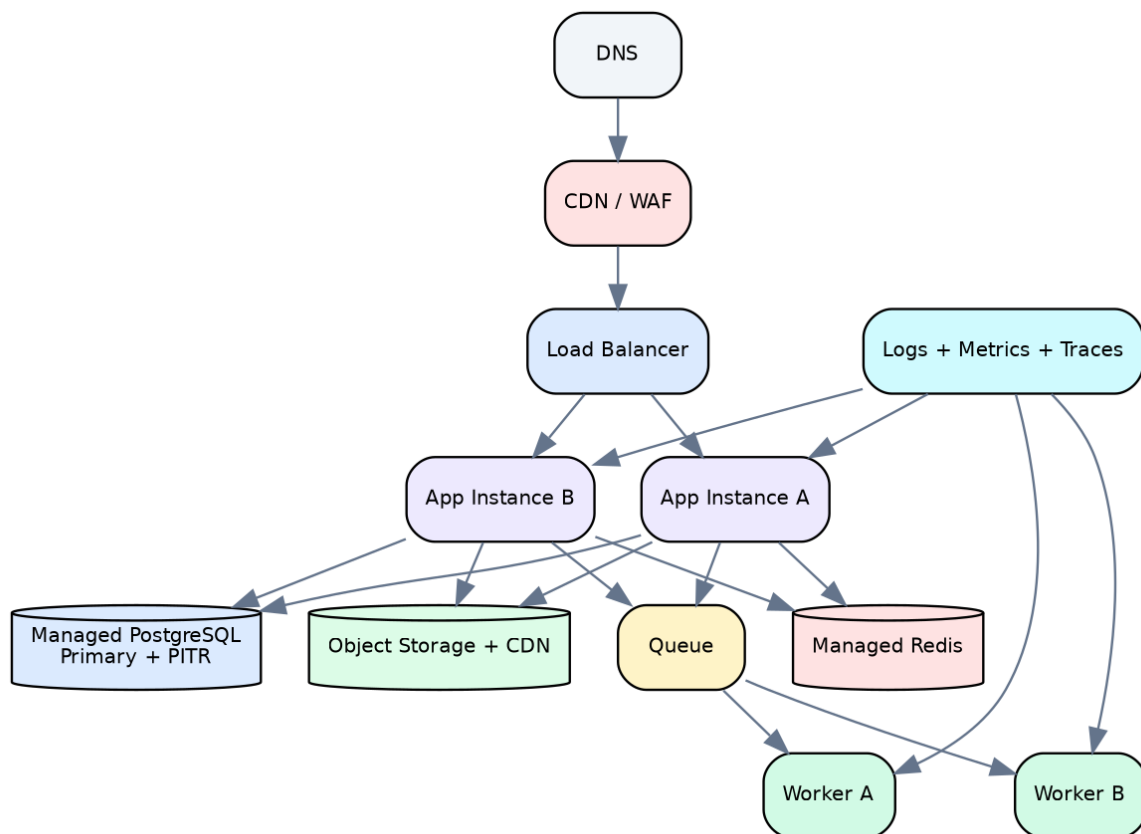
Role	Scope
Consumer member	own family permissions only
Family admin	invite/remove members, manage sharing and selected household settings
Acharya	own profile, availability, assigned bookings, earnings
Partner store	own store inventory/orders/staff
Temple staff	own tenant catalog, slots, orders, reports
Collection partner	assigned pickups/routes only
Support	masked customer data where possible; ticket-scoped access
Content reviewer	knowledge objects and review workflow; no finance access
Finance	ledger, settlement, reconciliation; restricted PII
Super admin	break-glass privileged actions with MFA and audit

Security Baseline

- TLS everywhere; HSTS; secure cookies; short-lived access tokens; refresh-token rotation.
- OTP and login rate limits by IP, device and account; fraud scoring for repeated failures.
- MFA mandatory for staff, temples, partners and Acharyas handling sensitive operations.

- Parameterized database access, strict validation, CSP, CSRF protection where cookie auth is used, file scanning and MIME verification.
- Signed URLs for private media; no public bucket exposure.
- Secrets in a secrets manager; never in repositories or mobile applications.
- Daily backups plus tested restore drills; disaster-recovery runbook and defined RPO/RTO.

19. Deployment Architecture



Environment Strategy

Environment	Purpose	Rules
Local	developer productivity	seed data, mocked providers
Development	shared integration	frequent deploys; synthetic data only
Staging	production-like QA/UAT	masked/synthetic data; provider sandboxes
Production	customer traffic	change approvals, backups, monitoring, least privilege
DR	recovery capability	restore procedure tested on schedule

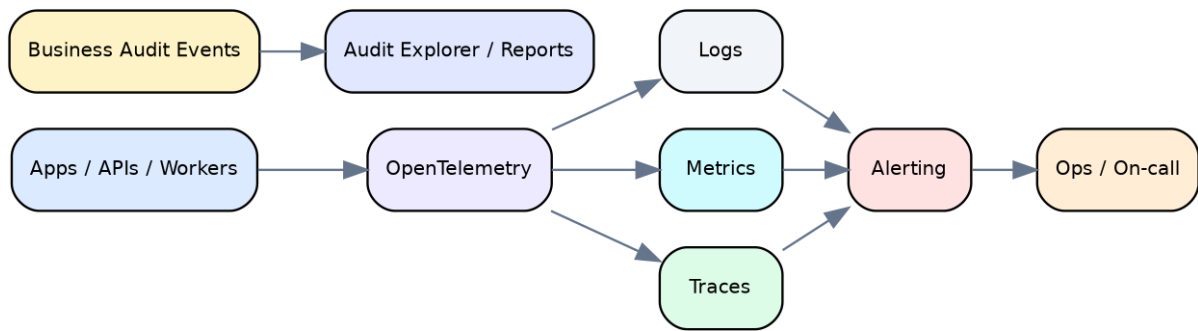
20. Scalability, Reliability & Performance

Concern	Design
Festival traffic spikes	CDN static caching, queue non-critical jobs, autoscaling/stateless app tier, precompute recommendations
Inventory races	database transaction + reservation token + Redis short lock
Acharya slot double booking	unique constraint on provider/time reservation and transaction lock
Temple seva capacity	atomic counter/row lock; idempotent booking confirmation
Payment duplicate webhook	idempotency key and processed-event table
Slow AI	stream answer; retrieve small approved context; cache repeated public knowledge
Media growth	object storage lifecycle tiers; thumbnails; CDN
Notification spikes	bulk queue, channel throttling, provider retry/fallback
Database growth	indexes, archival, read replicas, partition high-volume event/audit tables
Partner outage	circuit breaker, queue retry, graceful pending state

Performance Targets for V1

Path	Target
Normal authenticated API	p95 under 500 ms excluding external providers
Catalog browse	p95 under 300 ms from cache/search
Checkout internal processing	under 1 s before gateway redirect/SDK
SacredBox top-up calculation	under 2 s for normal household
VEDA first streamed token	target 2-4 s depending on provider
Admin list pages	under 2 s with pagination and server-side filters

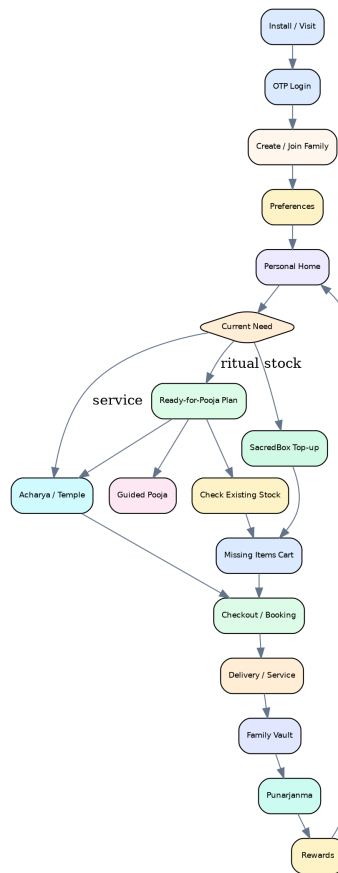
21. Observability, Audit & Operations



Business Audit Events

- Who changed ritual knowledge, what changed, reviewer and published version.
- Who changed product certification/trust status.
- Who created/cancelled/rescheduled a booking and why.
- Payment/refund/settlement transitions.
- Partner/temple staff permission changes.
- AI tool action request, user confirmation and resulting platform action.
- Vault media access/export/delete actions for sensitive family content.

22. End-to-End Master User Flow



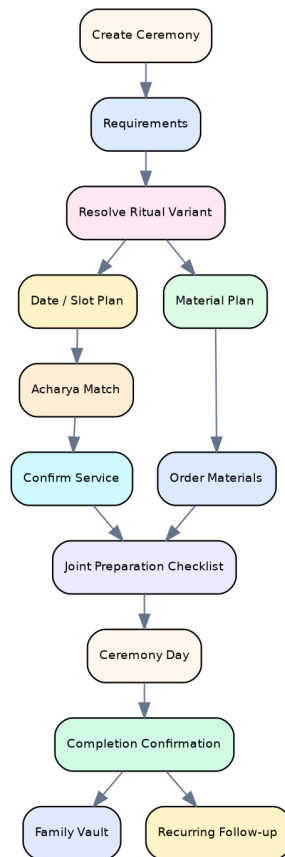
23. Detailed Workflow: Festival Preparation



Exception Paths

- Missing item unavailable: offer only rule-approved substitution or remove with clear warning.
- Acharya unavailable: show alternate verified providers or DIY guided mode if the ritual supports it.
- Late preparation: switch to express fulfillment only when available; never silently degrade the ritual plan.
- User already has material: one-tap "I have this" updates SacredBox estimate and removes the item.

24. Detailed Workflow: Lifecycle Ceremony



25. API Architecture & Integration Contracts

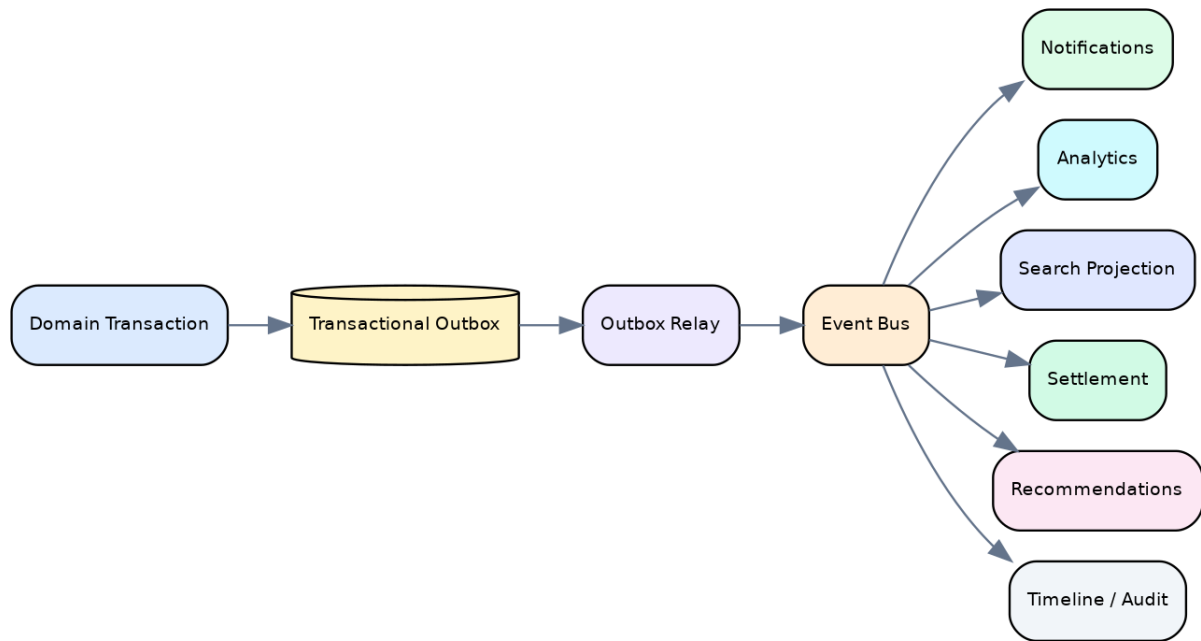
Expose stable domain APIs behind the BFF. Mobile/web clients should not directly depend on payment, WhatsApp, mapping or AI provider APIs. Provider-specific payloads are translated by adapters.

Domain	Representative endpoints
Identity	POST /auth/otp/request, /auth/otp/verify, GET /me
Families	/families, /families/{id}/members, /preferences
Ritual	/rituals, /rituals/{id}/variants, /ritual-plans/resolve
Calendar	/family-events, /calendar/upcoming, /reminders
Catalog	/products, /kits, /trust/{batch}
SacredBox	/boxes, /compartments, /consumption-events, /topup-plan
Cart/Orders	/cart, /checkout, /orders, /orders/{id}/cancel
Acharya	/acharyas/search, /availability, /bookings
Temple	/temples, /sevas, /seva-bookings
Punarjanma	/return-bags, /pickups, /impact
Finance	/payment-intents, /refunds, /wallet, provider webhooks
VEDA	/assistant/conversations, /assistant/messages, /assistant/actions/{id}/confirm
Admin	/admin/* with strict role-specific controllers

Webhook Principles

- Every external webhook is authenticated (signature/secret), timestamp checked where supported, stored raw for audit, and processed idempotently.
- Publish internal events after transaction commit using an outbox table. A worker relays the outbox to BullMQ/event bus.
- Integrations implement retry with exponential backoff and dead-letter handling.

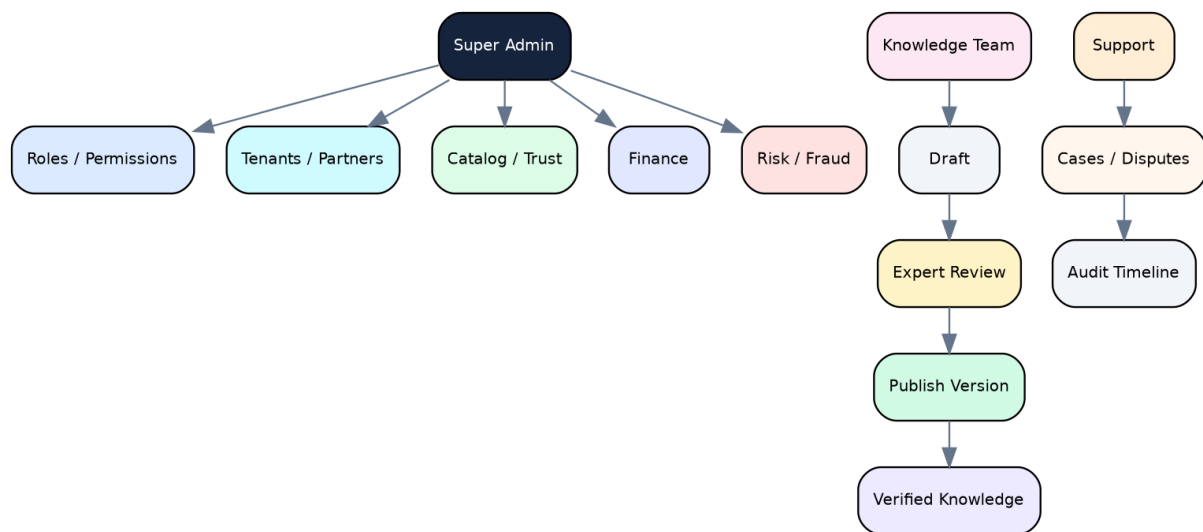
26. Event-Driven Architecture



Representative Events

Event	Consumers
family.event.created	calendar scheduler, recommendation engine
ritual.plan.resolved	cart suggestion, guide preloader, analytics
order.paid	fulfillment, invoice, notification, ledger
booking.confirmed	Acharya app, family reminders, finance
booking.completed	settlement, rating request, Vault
sacredbox.stock.low	top-up recommendation
return_bag.accepted	reward wallet, impact analytics
knowledge.version.published	search index, AI knowledge index, cache invalidation

27. Admin & Governance Architecture



Admin Modules

- Dashboard: GMV, active families, subscription retention, booking success, festival demand, fulfillment SLA.
- Family/customer support: account lookup, masked data, consent-aware impersonation or assisted support.
- Catalog & trust: products, SKUs, batches, certificates, approved substitutions, recall tools.
- Acharya governance: onboarding, KYC, skill verification, complaint history, availability health.
- Temple & partner tenancy: contracts, staff, catalogs, settlement and SLA.
- Ritual CMS: author -> reviewer -> publisher workflow; multilingual versioning.
- Finance: payments, refunds, commission, partner statements, reconciliation and exceptions.
- Punarjanma: pickup network, facility lots, yield, rejected contamination, impact reporting.
- Risk: suspicious accounts, coupon abuse, payment anomalies, fake reviews, KYC flags.

28. Analytics & KPI Architecture

Area	Core KPIs
Acquisition	CAC, organic/direct mix, onboarding completion, family creation
Engagement	WAU/MAU, calendar opens, VEDA helpfulness, guided ritual completion
Commerce	conversion, AOV, repeat rate, refill rate, kit attachment, fulfillment SLA
Subscription	trial conversion, monthly retention, pause/cancel reasons, LTV
Marketplace	search-to-book, fill rate, cancellations, completion, repeat Acharya rate
Temple	seva conversion, capacity utilization, prasadam SLA, repeat followers
SacredBox	scan adoption, top-up acceptance, stockout prevention estimate
Punarjanma	kg accepted, contamination, pickup success, reward redemption, output yield
Trust	complaint rate, content correction rate, product batch incidents
AI	grounded-answer rate, tool-action success, fallback/escalation, latency and cost

Data Pipeline

Product events should follow a documented event taxonomy. Initially, events can be appended to PostgreSQL or streamed to a lightweight analytics destination. At scale, stream events to object storage/warehouse and build transformation models for dashboards, cohort analysis and forecasting. Production PII should not be copied into analytics unless necessary; prefer pseudonymous family/user keys.

29. Delivery Roadmap

Phase	Scope	Exit Criteria
MVP - 0 to 4 months	Family account, ritual catalog, calendar, product commerce, basic SacredBox, guided online ritual, in-person, materials, follow guidance and store com	Guided online ritual, in-person, materials, follow guidance and store com
V1 - 4 to 8 months	Acharya marketplace, partner stores, parent care, subscription/refill, stronger RELS, governing and recurring household commerce	RELS, governing and recurring household commerce
V2 - 8 to 14 months	Temple tenants, VEDA RAG + safe actions, Punarjanma pilots, NRI features	Multi-sided platform operating in selected cities
V3 - 14 to 24 months	Advanced forecasting, premium SacredBox hardware, kiosks, multi-city recycling, national expansion, private/public, unit economics	National expansion, private/public, unit economics

MVP Do-Not-Build List

- Do not start with custom electronic SacredBox hardware; validate replenishment behavior first.
- Do not build microservices simply for architecture fashion.
- Do not build a general astrology platform unless it has a clearly validated role in the core journey.
- Do not allow open community-authored ritual instructions without moderation and expert review.
- Do not onboard every temple or every city at once; prove partner operations in one dense geography.
- Do not collect sacred waste categories before processing and compliance SOPs are validated.

30. Recommended Repository & Engineering Structure

```
aaradhya-one/  
  apps/  
    consumer-web/      # Next.js  
    admin-web/         # Next.js  
    partner-web/       # Next.js  
    temple-web/        # Next.js  
    mobile/            # Flutter  
    api/               # NestJS modular backend  
    worker/            # BullMQ/background jobs  
  packages/  
    contracts/         # OpenAPI/types/events  
    design-system/     # shared web UI tokens/components  
    config/            # lint/tsconfig/env validation  
  infra/  
    docker/  
    terraform-or-pulumi/  
    monitoring/  
  docs/  
    adr/               # architecture decision records  
    api/  
    threat-models/  
    runbooks/
```

CI/CD Quality Gates

- Lint, format and TypeScript/static checks.
- Unit tests for domain rules; integration tests with disposable PostgreSQL/Redis.
- API contract tests and migration checks.
- Dependency and secret scanning.
- Container/image vulnerability scan where containers are used.
- Staging deploy, smoke tests and manual approval for high-risk production changes.
- Database migrations are backward-compatible for rolling deployment where possible.

31. Mermaid Source Appendix

The following Mermaid-compatible source definitions correspond to the architecture diagrams in this blueprint. The PDF renders colorful equivalents for reliable offline viewing. These definitions can be copied into Mermaid Live Editor, Markdown environments with Mermaid support, or project documentation.

1. 01_platform_loop

```
flowchart TD
    F[Family] --> DP[Family Dharma Profile]
    DP --> C[Spiritual Calendar]
    C --> R[Ritual Orchestrator]
    R --> K[Custom Kit]
    R --> A[Acharya Booking]
    R --> T[Temple / Seva]
    R --> G[Guided Pooja]
    K --> S[SacredBox]
    G --> V[Family Vault]
    S --> P[Punarjanma Return]
    P --> RW[Rewards]
    RW --> C
```

2. 02_experience_arch

```
flowchart LR
    subgraph Consumer
        APP[Flutter Mobile]
        WEB[Next.js Web]
        VEDA[VEDA Assistant]
    end
    subgraph ServicePartners
        ACH[Acharya App]
        VEN[Partner Store App]
        TEM[Temple Portal]
        REC[Collection Partner]
    end
    subgraph Enterprise
        ADM[Super Admin]
        OPS[Ops / Support]
        CMS[Knowledge CMS]
    end
    Consumer --> API[Experience APIs]
    ServicePartners --> API
    Enterprise --> API
```

3. 03_domains

```
flowchart TB
    CORE[Platform Core] --> ID[Identity & Family]
    CORE --> RIT[Ritual Knowledge]
    CORE --> CAL[Calendar & Events]
    CORE --> COM[Commerce]
    CORE --> MKT[Marketplace]
    CORE --> TMP[Temple]
    CORE --> BOX[SacredBox]
    CORE --> CIR[Punarjanma]
    CORE --> AI[VEDA AI]
    CORE --> FIN[Payments & Ledger]
    CORE --> MSG[Notifications]
    CORE --> GOV[Trust & Governance]
```

4. 04_software_arch

```
flowchart TB
    C[Clients] --> CDN[CDN / WAF]
    CDN --> BFF[API Gateway / BFF]
    BFF --> APP[Modular Application]
    APP --> PG[(PostgreSQL)]
    APP --> REDIS[(Redis)]
    APP --> OBJ[(Object Storage)]
    APP --> QUEUE[Queue / Event Bus]
    QUEUE --> WORK[Workers]
    WORK --> EXT[Payments / Messaging / Maps / AI]
    APP --> SEARCH[(Search Index optional)]
```

5. 05_internal_layers

```
flowchart LR
    API[Controllers / BFF] --> APP[Application Services]
    APP --> DOM[Domain Layer]
    DOM --> REP[Repositories]
    REP --> DB[(PostgreSQL)]
    APP --> EVT[Domain Events]
    EVT --> WRK[Workers]
    APP --> ADP[Adapters]
    ADP --> EXT[External Providers]
```

6. 06_data_model

```
erDiagram
    FAMILY ||--o{ FAMILY_MEMBER : contains
    FAMILY ||--o{ FAMILY_EVENT : plans
    FAMILY ||--o{ SACREDBOX : owns
    RITUAL ||--o{ RITUAL_VARIANT : has
    RITUAL_VARIANT ||--o{ RITUAL_REQUIREMENT : requires
    PRODUCT ||--o{ PRODUCT_VARIANT : has
    FAMILY_EVENT }o--|| RITUAL_VARIANT : uses
    ORDER ||--o{ ORDER_ITEM : contains
    FAMILY ||--o{ ORDER : places
    ACHARYA ||--o{ BOOKING : receives
    FAMILY ||--o{ BOOKING : creates
    TEMPLE ||--o{ SEVA : offers
    FAMILY ||--o{ VAULT_ENTRY : stores
    FAMILY ||--o{ RETURN_BAG : returns
```

7. 07_profile_engine

```
flowchart TD
  ONB[Onboarding] --> PREF[Explicit Preferences]
  PREF --> FAM[Family Profile]
  FAM --> CAL[Calendar Builder]
  HIST[Order + Ritual History] --> REC[Recommendation Engine]
  CAL --> REC
  FAM --> REC
  REC --> READY[Ready-for-Pooja Plan]
  READY --> KIT[Kit]
  READY --> ACH[Acharya]
  READY --> GUIDE[Guidance]
  READY --> REM[Reminder]
```

8. 08_rules_engine

```
flowchart LR
  R[Ritual Master] --> V[Variant]
  V --> C{Context Rules}
  C --> L[Language]
  C --> REG[Region / Tradition]
  C --> MODE[DIY / Acharya]
  C --> CNT[Participant Count]
  L --> PLAN[Ritual Plan]
  REG --> PLAN
  MODE --> PLAN
  CNT --> PLAN
  PLAN --> MAT[Material List]
  PLAN --> STEP[Guided Steps]
  PLAN --> SERV[Service Needs]
```

9. 09_sacredbox

```
flowchart TD
  BOX[SacredBox] --> SCAN[QR/NFC Scan]
  SCAN --> EVT[Consumption Event]
  EVT --> STOCK[Estimated Stock]
  HIST[Purchase History] --> FORE[Depletion Forecast]
  STOCK --> FORE
  CAL[Upcoming Calendar] --> NEED[Upcoming Requirements]
  FORE --> NEED
  NEED --> GAP[Gap Analysis]
  GAP --> TOPUP[Smart Top-up Cart]
  TOPUP --> USER{User Approval}
  USER -->|Yes| ORDER[Order]
  USER -->|No| SNOOZE[Snooze / Edit]
```

10. 10_commerce

```
flowchart LR
  U[User] --> CART[Cart]
  CART --> PRICE[Pricing / Offers]
  PRICE --> INV[Inventory Reservation]
  INV --> PAY[Payment]
  PAY --> ORD[Confirmed Order]
  ORD --> ROUTE[Fulfillment Router]
  ROUTE --> WH[Warehouse]
  ROUTE --> PS[Partner Store]
  ROUTE --> TMP[Temple]
  WH --> DEL[Delivery]
  PS --> DEL
  TMP --> DEL
  DEL --> DONE[Delivered]
  DONE --> LEDGER[Settlement Ledger]
```

11. 11_acharya

```
flowchart TD
  REQ[Family Ceremony Request] --> MATCH[Matching Engine]
  PROF[Verified Acharya Profiles] --> MATCH
  AV[Availability] --> MATCH
  LOC[Distance / Remote Eligibility] --> MATCH
  MATCH --> LIST[Ranked Options]
  LIST --> SLOT[Slot Lock]
  SLOT --> PAY[Booking / Payment]
  PAY --> PREP[Preparation Checklist]
  PREP --> SERV[Service]
  SERV --> OTP[Completion OTP / Confirmation]
  OTP --> SET[Settlement]
  OTP --> RATE[Rating / Family Favorite]
```

12. 12_temple

```
flowchart LR
  T[Tenant: Temple] --> CAT[Seva / Prasadam Catalog]
  CAT --> CAP[Slot / Capacity]
  CAP --> API[Platform API]
  U[Family] --> API
  API --> BOOK[Booking / Order]
  BOOK --> PAY[Payment]
  PAY --> FUL[Fulfillment / Seva Execution]
  FUL --> PROOF[Receipt / Prasadam / Confirmation]
  PROOF --> SET[Temple Settlement]
  PROOF --> VAULT[Family Vault]
```

13. 13_veda

```
flowchart TB
    U[User] --> UI[VEDA UI]
    UI --> ORCH[AI Orchestrator]
    ORCH --> INT[Intent + Safety]
    INT --> RAG[Retrieve Approved Knowledge]
    RAG --> KB[(Verified Knowledge Base)]
    INT --> TOOL{Action Needed?}
    TOOL -->|No| LLM[Response Generator]
    TOOL -->|Yes| POL[Tool Policy / Confirmation]
    POL --> ACT[Platform APIs]
    ACT --> LLM
    RAG --> LLM
    LLM --> CIT[Answer + Source/Action Trace]
```

14. 14_parent_care

```
flowchart LR
    P[Payer / Child] --> PLAN[Care Plan]
    PLAN --> FAM[Parent Family Profile]
    FAM --> CAL[Calendar]
    CAL --> NEED[Needs Engine]
    NEED --> DEL[Auto-prepared Deliveries]
    NEED --> SERV[Optional Services]
    DEL --> PAR[Parents]
    SERV --> PAR
    PAR --> ACK[Delivery / Completion]
    ACK --> P
```

15. 15_punarjanma

```
flowchart TD
    HOME[Household / Temple] --> BAG[Eligible Material Bag]
    BAG --> PICK[Pickup / Drop]
    PICK --> SCAN[Bag Scan + Weight]
    SCAN --> SORT[Sorting / Contamination Check]
    SORT --> LOT[Processing Lot]
    LOT --> OUT[Compost / Dhoop / Paper / Other]
    OUT --> QA[Quality / Compliance]
    QA --> PROD[New Product Batch]
    PROD --> STORE[Aaradhya Store]
    SCAN --> REW[Impact + Reward]
    REW --> HOME
```

16. 16_finance

```
flowchart LR
  C[Checkout] --> PI[Payment Intent]
  PI --> PG[Payment Gateway]
  PG --> WH[Webhook]
  WH --> ORD[Order / Booking State]
  WH --> LED[Immutable Ledger]
  LED --> COMM[Commission]
  LED --> PART[Partner Payable]
  LED --> TAX[Tax / Invoice]
  LED --> REF[Refunds]
  LED --> REW[Reward Wallet]
```

17. 17_notifications

```
flowchart TD
  EVT[Domain Event] --> N[Notification Orchestrator]
  N --> PREF[Consent + Preferences]
  PREF --> TMP[Template Renderer]
  TMP --> ROUTE{Best Channel}
  ROUTE --> PUSH[Push]
  ROUTE --> WA[WhatsApp]
  ROUTE --> SMS[SMS]
  ROUTE --> EMAIL[Email]
  PUSH --> STAT[Delivery Status]
  WA --> STAT
  SMS --> STAT
  EMAIL --> STAT
  STAT --> RETRY[Retry / Fallback Policy]
```

18. 18_security

```
flowchart TB
  USER[Users / Staff / Partners] --> WAF[WAF + Bot / Rate Limit]
  WAF --> AUTH[Identity Service]
  AUTH --> MFA[MFA / Device Trust]
  AUTH --> API[API Authorization]
  API --> RBAC[RBAC + Tenant Scope]
  RBAC --> DATA[(Encrypted Data)]
  API --> AUD[Audit Log]
  DATA --> BACK[Encrypted Backups]
  API --> SEC[Security Monitoring]
  SEC --> IR[Incident Response]
```


19. 19_deployment

```
flowchart TB
    DNS[DNS] --> CDN[CDN / WAF]
    CDN --> LB[Load Balancer]
    LB --> APP1[App Instance A]
    LB --> APP2[App Instance B]
    APP1 --> PG[(Managed PostgreSQL)]
    APP2 --> PG
    APP1 --> REDIS1[(Managed Redis)]
    APP2 --> REDIS1
    APP1 --> OBJ[(Object Storage)]
    APP2 --> OBJ
    APP1 --> Q1[Queue]
    APP2 --> Q1
    Q1 --> W1[Worker A]
    Q1 --> W2[Worker B]
    MON[Monitoring] --> APP1
    MON --> APP2
    MON --> W1
    MON --> W2
```

20. 20_observability

```
flowchart LR
    APP[Apps / APIs / Workers] --> OTEL[OpenTelemetry]
    OTEL --> LOG[Central Logs]
    OTEL --> MET[Metrics]
    OTEL --> TR[Traces]
    LOG --> ALERT[Alerting]
    MET --> ALERT
    TR --> ALERT
    ALERT --> OPS[Ops / On-call]
    AUD[Business Audit Events] --> BI[Audit Explorer / Reports]
```

21. 21_master_flow

```
flowchart TD
    START[Install / Visit] --> LOGIN[OTP Login]
    LOGIN --> FAM[Create / Join Family]
    FAM --> PREF[Select Preferences]
    PREF --> HOME[Personal Home]
    HOME --> EVT{Need}
    EVT -->|Festival / Ritual| PLAN[Ready-for-Pooja Plan]
    EVT -->|Routine stock| BOX[SacredBox Top-up]
    EVT -->|Service| ACH[Acharya / Temple]
    PLAN --> CHECK[Check Existing Stock]
    CHECK --> CART[Only Missing Items]
    PLAN --> GUIDE[Guided Pooja]
    PLAN --> ACH
    CART --> PAY[Checkout]
    ACH --> PAY
    PAY --> FUL[Delivery / Service]
    FUL --> VAULT[Family Vault]
    VAULT --> RETURN[Punarjanma Return]
    RETURN --> REWARD[Rewards]
    REWARD --> HOME
```

22. 22_festival_flow

```
sequenceDiagram
    participant C as Calendar
    participant R as Ritual Engine
    participant B as SacredBox
    participant I as Inventory
    participant U as User
    C->>R: Preparation window starts
    R->>B: Required materials vs family stock
    B->>R: Estimated available quantities
    R->>I: Check local stock & lead times
    I->>R: Availability
    R->>U: Ready-for-Pooja plan
    U->>R: Approve/edit
    R->>I: Reserve missing items
    U->>R: Pay
    R->>U: Delivery + guided ritual + service reminders
```

23. 23_lifecycle

```
flowchart TD
    U[Create Ceremony] --> REQ[Collect Requirements]
    REQ --> VAR[Resolve Ritual Variant]
    VAR --> DATE[Date / Slot Planning]
    DATE --> ACH[Acharya Matching]
    VAR --> KIT[Material Plan]
    ACH --> CONF[Confirm Service]
    KIT --> ORDER[Order / Source Materials]
    CONF --> PREP[Joint Preparation Checklist]
    ORDER --> PREP
    PREP --> DAY[Ceremony Day]
    DAY --> DONE[Completion]
    DONE --> VAULT[Vault + Media + Invoice]
    DONE --> REC[Recurring Follow-up if Applicable]
```

24. 24_events

```
flowchart LR
    DOM[Domain Transaction] --> OUT[(Outbox)]
    OUT --> RELAY[Outbox Relay]
    RELAY --> BUS[Event Bus]
    BUS --> NOTIF[Notifications]
    BUS --> ANALYTICS[Analytics]
    BUS --> SEARCH[Search Projection]
    BUS --> FIN[Settlement Worker]
    BUS --> AI[Recommendation Features]
    BUS --> AUD[Audit / Timeline]
```

25. 25_governance

```
flowchart TB
    SA[Super Admin] --> IAM[Roles / Permissions]
    SA --> TEN[Tenant Governance]
    SA --> CAT[Catalog / Trust]
    SA --> FIN[Finance]
    SA --> RISK[Risk / Fraud]
    CMS[Knowledge Team] --> DRAFT[Draft]
    DRAFT --> REV[Expert Review]
    REV --> PUB[Publish Version]
    PUB --> KB[Verified Knowledge]
    OPS[Support] --> TKT[Cases / Disputes]
    TKT --> AUD[Audit Timeline]
```

32. Final Architecture Recommendation

Build Aaradhya One as a trusted orchestration platform first, not as a giant marketplace. The winning system is the one that understands a family's explicitly chosen devotional routines, prepares only what is needed, connects verified services, preserves family history, and closes the loop through responsible reuse.

The architecture is intentionally modular, provider-independent and phased. It supports a low-cost MVP while leaving clear migration paths toward high-volume commerce, temple tenants, AI assistance, IoT-enabled SacredBox variants, national partner operations and circular-material traceability.

Recommended First Release

Priority	Build
P0	Identity, family profile, addresses, preferences, consent
P0	Ritual/variant CMS + verified guided content
P0	Calendar + preparation reminders
P0	Catalog, kit resolver, cart, checkout, payments, order fulfillment
P0	Basic SacredBox stock estimate + top-up
P0	Admin, audit, support and notifications
P1	Acharya marketplace + booking
P1	Parent care + membership/refill
P2	VEDA grounded assistant
P2	Temple tenants + Punarjanma pilot

This sequencing maximizes learning: first prove that households repeatedly use personalized planning and replenishment. Then add marketplace and institutional layers around that retained family relationship.